📘 Strings in Python

String Operations | All Built-in String Methods/Functions

Introduction to Strings

Definition: A string is a sequence of characters enclosed in single quotes (`' '`) or double quotes (`" "`).

```
str1 = 'Hello'
str2 = "World"
str3 = '''Multi-line
string'''
```

Important Points:

- Strings are immutable in Python (cannot be changed after creation)
- Characters in a string can be accessed using index positions
- Indexing starts from 0 (forward) and -1 (backward)

String Operations

1. Concatenation (+)

Purpose: Joins two or more strings together

Syntax: `string1 + string2`

Parameters: Two or more strings

Return: A new string combining all strings

- Both operands must be strings
- Creates a new string (doesn't modify original)

```
str1 = "Hello"
str2 = "World"
result = str1 + " " + str2  # "Hello World"
```

2. Repetition / Replication (*)

Purpose: Repeats a string a specified number of times

Syntax: `string * n` or `n * string`

Parameters: string (compulsory), integer n (compulsory) — number of repetitions

Return: A new string repeated n times

- n must be an integer
- If n = 0, returns empty string

```
str1 = "Hi"
result = str1 * 3  # "HiHiHi"
```

3. Indexing

Purpose: Accesses an individual character of a string using its position

Syntax: `string[index]`

Parameters: index (compulsory) — position of the character

Return: A single character at the given index

- Forward indexing starts from 0 (left to right)
- Backward indexing starts from -1 (right to left)
- Index must be an integer, otherwise `TypeError` occurs
- Raises `IndexError` if the index is out of range

```
str1 = "Hello"
print(str1[0])    # "H" (first character)
print(str1[4])    # "o" (last character)
print(str1[-1])   # "o" (last character, backward index)
print(str1[-5])   # "H" (first character, backward index)
# print(str1[10])  # IndexError: string index out of range
```

4. Slicing

Purpose: Extracts a portion of a string

Syntax: `string[start:end:step]`

Parameters:

- start (optional) — starting index (default: 0)
- end (optional) — ending index (excluded, default: length)
- step (optional) — step value (default: 1)

Return: A new substring

- All parameters are optional
- Negative indices can be used
- start < end for forward slicing

```
str1 = "Hello World"
print(str1[0:5])      # "Hello"
print(str1[6:])       # "World"
print(str1[::-1])     # "dlroW olleH" (reverse)
print(str1[::2])      # "HloWrd"
```

5. Membership Operators (in / not in)

Purpose: Checks if a substring exists in a string

Syntax: `substring in string` or `substring not in string`

Parameters: substring (compulsory) — string to search for

Return: Boolean (True/False)

- Case-sensitive
- `in` returns True if substring is present
- `not in` returns True if substring is not present

```
str1 = "Hello World"
print("World" in str1)       # True
print("world" in str1)       # False (case-sensitive)
print("Python" not in str1)  # True
```

Traversing a String Using Loops

Using for loop:

```
str1 = "Python"
for ch in str1:
    print(ch)
# Output: P y t h o n (each on new line)
```

Using for loop with index:

```
str1 = "Python"
for i in range(len(str1)):
    print(str1[i])
```

Using while loop:

```
str1 = "Python"
i = 0
while i < len(str1):
    print(str1[i])
    i += 1
```

Built-in Functions / Methods

1. len()

Purpose: Returns the length of a string

Syntax: `len(string)`

Parameters: string (compulsory)

Return: Integer (number of characters)

- Counts all characters including spaces

```
str1 = "Hello World"
print(len(str1))  # 11
```

2. capitalize()

Purpose: Converts the first character to uppercase and rest to lowercase

Syntax: `string.capitalize()`

Parameters: No parameters

Return: New string with first letter capitalized

- Only the first character is affected
- All other characters become lowercase
- Non-alphabetic first character remains unchanged

```
str1 = "hello WORLD"
print(str1.capitalize())  # "Hello world"
```

3. title()

Purpose: Converts first character of each word to uppercase

Syntax: `string.title()`

Parameters: No parameters

Return: New string with each word's first letter capitalized

- Every word's first letter becomes uppercase
- Remaining letters become lowercase

```
str1 = "hello world python"
print(str1.title())  # "Hello World Python"
```

4. lower()

Purpose: Converts all characters to lowercase

Syntax: `string.lower()`

Parameters: No parameters

Return: New string in lowercase

- Only affects alphabetic characters

```
str1 = "HeLLo WoRLD"
print(str1.lower())  # "hello world"
```

5. upper()

Purpose: Converts all characters to uppercase

Syntax: `string.upper()`

Parameters: No parameters

Return: New string in uppercase

- Only affects alphabetic characters

```
str1 = "Hello World"
print(str1.upper())  # "HELLO WORLD"
```

6. count()

Purpose: Counts occurrences of a substring in a string

Syntax: `string.count(substring, start, end)`

Parameters:

- substring (compulsory) — string to search for
- start (optional) — starting index (default: 0)
- end (optional) — ending index (default: length)

Return: Integer (number of occurrences)

- Case-sensitive search
- Returns 0 if substring not found
- Overlapping occurrences are not counted

```
str1 = "Hello Hello Hello"
print(str1.count("Hello"))        # 3
print(str1.count("Hello", 0, 10)) # 1
```

7. find()

Purpose: Finds the first occurrence of a substring

Syntax: `string.find(substring, start, end)`

Parameters:

- substring (compulsory) — string to search for

- start (optional) — starting index (default: 0)
- end (optional) — ending index (default: length)

Return: Integer (index of first occurrence) or -1 if not found

- Case-sensitive
- Returns -1 if substring not found
- Starts search from left to right

```
str1 = "Hello World World"
print(str1.find("World"))   # 6
print(str1.find("world"))   # -1
```

8. index()

Purpose: Finds the first occurrence of a substring

Syntax: `string.index(substring, start, end)`

Parameters:

- substring (compulsory) — string to search for
- start (optional) — starting index (default: 0)
- end (optional) — ending index (default: length)

Return: Integer (index of first occurrence)

- Case-sensitive
- Raises ValueError if substring not found
- Similar to find() but raises error instead of returning -1

```
str1 = "Hello World World"
print(str1.index("World"))   # 6
# print(str1.index("world"))   # ValueError
```

9. endswith()

Purpose: Checks if string ends with a specified suffix

Syntax: `string.endswith(suffix, start, end)`

Parameters:

- suffix (compulsory) — string or tuple of strings

- start (optional) — starting index (default: 0)
- end (optional) — ending index (default: length)

Return: Boolean (True/False)

- Case-sensitive
- Can check multiple suffixes using tuple

```
str1 = "Hello World"
print(str1.endswith("World"))   # True
print(str1.endswith("world"))   # False
print(str1.endswith(("World", "Python")))  # True
```

10. startswith()

Purpose: Checks if string starts with a specified prefix

Syntax: `string.startswith(prefix, start, end)`

Parameters:

- prefix (compulsory) — string or tuple of strings
- start (optional) — starting index (default: 0)
- end (optional) — ending index (default: length)

Return: Boolean (True/False)

- Case-sensitive
- Can check multiple prefixes using tuple

```
str1 = "Hello World"
print(str1.startswith("Hello"))   # True
print(str1.startswith("hello"))   # False
```

11. isalnum()

Purpose: Checks if all characters are alphanumeric (A-Z, a-z, 0-9)

Syntax: `string.isalnum()`

Parameters: No parameters

Return: Boolean (True/False)

- Returns False if string contains spaces or special characters

- Returns False if string is empty

```
str1 = "Hello123"
print(str1.isalnum())   # True
str2 = "Hello 123"
print(str2.isalnum())   # False (space)
```

12. isalpha()

Purpose: Checks if all characters are alphabetic (A-Z, a-z)

Syntax: `string.isalpha()`

Parameters: No parameters

Return: Boolean (True/False)

- Returns False if string contains digits, spaces, or special characters
- Returns False if string is empty

```
str1 = "Hello"
print(str1.isalpha())   # True
str2 = "Hello123"
print(str2.isalpha())   # False
```

13. isdigit()

Purpose: Checks if all characters are digits (0-9)

Syntax: `string.isdigit()`

Parameters: No parameters

Return: Boolean (True/False)

- Returns False if string contains letters, spaces, or special characters
- Returns False if string is empty

```
str1 = "12345"
print(str1.isdigit())   # True
str2 = "12345abc"
print(str2.isdigit())   # False
```

14. islower()

Purpose: Checks if all alphabetic characters are lowercase

Syntax: `string.islower()`

Parameters: No parameters

Return: Boolean (True/False)

- Ignores non-alphabetic characters
- Returns False if string contains uppercase letters

```
str1 = "hello world"
print(str1.islower())   # True
str2 = "Hello World"
print(str2.islower())   # False
```

15. isupper()

Purpose: Checks if all alphabetic characters are uppercase

Syntax: `string.isupper()`

Parameters: No parameters

Return: Boolean (True/False)

- Ignores non-alphabetic characters
- Returns False if string contains lowercase letters

```
str1 = "HELLO WORLD"
print(str1.isupper())   # True
str2 = "HELLO World"
print(str2.isupper())   # False
```

16. isspace()

Purpose: Checks if all characters are whitespace

Syntax: `string.isspace()`

Parameters: No parameters

Return: Boolean (True/False)

- Whitespace includes space, tab, newline, etc.

- Returns False if string is empty

```
str1 = "   "
print(str1.isspace())   # True
str2 = "  a  "
print(str2.isspace())   # False
```

17. lstrip()

Purpose: Removes leading whitespace characters

Syntax: `string.lstrip(chars)`

Parameters: chars (optional) — characters to remove (default: whitespace)

Return: New string with leading characters removed

- Removes from left side only
- Removes all occurrences of specified characters

```
str1 = "   Hello World   "
print(str1.lstrip())   # "Hello World   "
str2 = "***Hello***"
print(str2.lstrip("*"))   # "Hello***"
```

18. rstrip()

Purpose: Removes trailing whitespace characters

Syntax: `string.rstrip(chars)`

Parameters: chars (optional) — characters to remove (default: whitespace)

Return: New string with trailing characters removed

- Removes from right side only
- Removes all occurrences of specified characters

```
str1 = "   Hello World   "
print(str1.rstrip())   # "   Hello World"
str2 = "***Hello***"
print(str2.rstrip("*"))   # "***Hello"
```

19. strip()

Purpose: Removes leading and trailing whitespace characters

Syntax: `string.strip(chars)`

Parameters: chars (optional) — characters to remove (default: whitespace)

Return: New string with leading and trailing characters removed

- Removes from both left and right sides
- Removes all occurrences of specified characters

```
str1 = "   Hello World   "
print(str1.strip())   # "Hello World"
str2 = "***Hello***"
print(str2.strip("*"))   # "Hello"
```

20. replace()

Purpose: Replaces all occurrences of a substring with another substring

Syntax: `string.replace(old, new, count)`

Parameters:

- old (compulsory) — substring to be replaced
- new (compulsory) — replacement substring
- count (optional) — number of replacements (default: all)

Return: New string with replacements made

- Case-sensitive
- Returns a new string (doesn't modify original)

```
str1 = "Hello Hello Hello"
print(str1.replace("Hello", "Hi"))        # "Hi Hi Hi"
print(str1.replace("Hello", "Hi", 2))     # "Hi Hi Hello"
```

21. join()

Purpose: Joins elements of a sequence (list/tuple) with a separator

Syntax: `separator.join(sequence)`

Parameters: sequence (compulsory) — iterable of strings

Return: A string with all elements joined

- All elements must be strings
- Separator string comes before .join()

```
words = ["Hello", "World", "Python"]
separator = " "
result = separator.join(words)
print(result)  # "Hello World Python"
```

22. partition()

Purpose: Splits the string at the first occurrence of a separator

Syntax: `string.partition(separator)`

Parameters: separator (compulsory) — string to split at

Return: Tuple of 3 strings: (before, separator, after)

- Always returns a tuple of 3 strings
- If separator not found: (string, "", "")

```
str1 = "Hello-World-Python"
result = str1.partition("-")
print(result)  # ("Hello", "-", "World-Python")
```

23. split()

Purpose: Splits the string into a list of substrings

Syntax: `string.split(separator, maxsplit)`

Parameters:

- separator (optional) — delimiter (default: whitespace)
- maxsplit (optional) — maximum number of splits (default: all)

Return: List of substrings

- Returns a list
- If separator not specified, splits on whitespace

```
str1 = "Hello World Python"
print(str1.split())            # ["Hello", "World", "Python"]
print(str1.split(" ", 1))      # ["Hello", "World Python"]
```

Quick Reference Table

Method	Purpose	Returns
`len()`	String length	Integer
`capitalize()`	Capitalize first letter	String
`title()`	Capitalize each word	String
`lower()`	Convert to lowercase	String
`upper()`	Convert to uppercase	String
`count()`	Count occurrences	Integer
`find()`	Find substring	Integer
`index()`	Find substring	Integer
`endswith()`	Check suffix	Boolean
`startswith()`	Check prefix	Boolean
`isalnum()`	Check alphanumeric	Boolean
`isalpha()`	Check alphabetic	Boolean
`isdigit()`	Check digits	Boolean
`islower()`	Check lowercase	Boolean
`isupper()`	Check uppercase	Boolean
`isspace()`	Check whitespace	Boolean
`lstrip()`	Remove left whitespace	String
`rstrip()`	Remove right whitespace	String
`strip()`	Remove both whitespace	String
`replace()`	Replace substring	String

Method	Purpose	Returns
`join()`	Join with separator	String
`partition()`	Split at first occurrence	Tuple
`split()`	Split into list	List

Important Board Exam Tips Must Know

1. **String Immutability:** Strings cannot be changed in place. All operations return a new string.
2. **Indexing:** Remember that indexing starts from 0 in Python.
3. **Negative Indexing:** -1 is the last character, -2 is second last, etc.
4. **Slicing Rules:**
   - `string[start:end]` — includes start, excludes end
   - `string[:end]` — from beginning to end-1
   - `string[start:]` — from start to end
   - `string[::-1]` — reverses the string
5. **Membership Operators:** `in` and `not in` are case-sensitive.
6. **count() vs find() vs index():**
   - `count()` : Returns number of occurrences
   - `find()` : Returns index or -1
   - `index()` : Returns index or raises error
7. **strip() vs lstrip() vs rstrip():**
   - `strip()` : Removes from both ends
   - `lstrip()` : Removes from left only
   - `rstrip()` : Removes from right only
8. **join() vs split():**
   - `join()` : List → String
   - `split()` : String → List
9. **partition() vs split():**
   - `partition()` : Returns tuple of 3 elements
   - `split()` : Returns list of multiple elements

10. **Common Errors to Avoid:**
    - Forgetting that strings are immutable
    - Using wrong index positions
    - Case sensitivity issues
    - Forgetting to import modules (not needed for these methods)
